
CHAPTER 6

LINEAR CLASSIFIER AND LOGISTIC REGRESSION

NOTE: This chapter is written based on chapter 19.6.4 and 19.6.5 of *Artificial Intelligence A Modern Approach* by Stuart Russel and Peter Norvig

6.1 Linear Functions with Thresholds for Classification

Linear functions can also be used for classification tasks. In this case, the linear equation is used to define a boundary that separates the two classes. This linear equation is called the **Decision Boundary**. A linear decision boundary, also known as a linear separator, is used to separate data into classes. If a dataset can be perfectly divided by such a boundary, it is called linearly separable.

Our reference book shows an example of classification between two types of seismic events:

- Earthquakes, which interest seismologists, labeled 0.
- underground explosions, which concern arms control experts, labeled 1.

The classification is dependent on two input features

- The magnitudes of body, x_1 .
- surface waves from the seismic signal, x_2

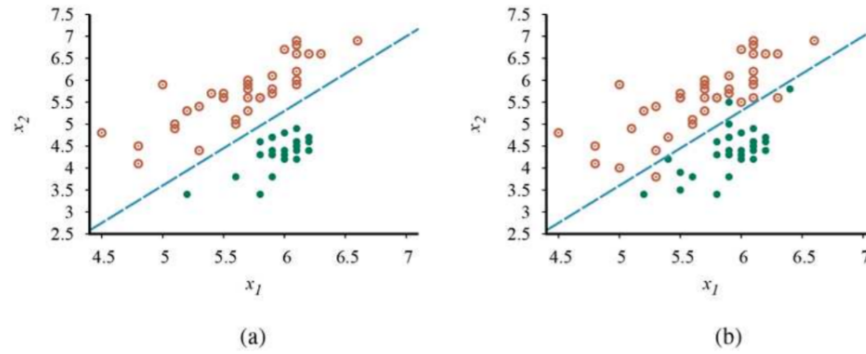
The goal is to learn a hypothesis, h that can correctly classify new points, (x_1, x_2) , labeling earthquakes as 0 and explosions as 1. The linear separator for this example dataset is

$$-4.9 + 1.7x_1 + x_2 = 0$$

From the plots we can see that the data points for explosions are below the line, meaning

$$-4.9 + 1.7x_1 + x_2 > 0$$

Figure 19.15



(a) Plot of two seismic data parameters, body wave magnitude x_1 and surface wave magnitude x_2 , for earthquakes (open orange circles) and nuclear explosions (green circles) occurring between 1982 and 1990 in Asia and the Middle East (Kebeasy *et al.*, 1998). Also shown is a decision boundary between the classes. (b) The same domain with more data points. The earthquakes and explosions are no longer linearly separable.

and the data points for earthquakes are above the line, meaning

$$-4.9 + 1.7x_1 + x_2 < 0$$

In Linear Classification with threshold we want to develop an equation for the decision boundary.

$$Z = \mathbf{W} \cdot \mathbf{X} + W_0 \quad (6.1)$$

by predicting the weights, $\mathbf{W}$ and W_0 . This equation is then used to construct the hypothesis,

$$\mathbf{h}_{\mathbf{W}}(\mathbf{x}) = \begin{cases} 1 & \text{if } Z > 0; \\ 0 & \text{if } Z < 0 \end{cases} \quad (6.2)$$

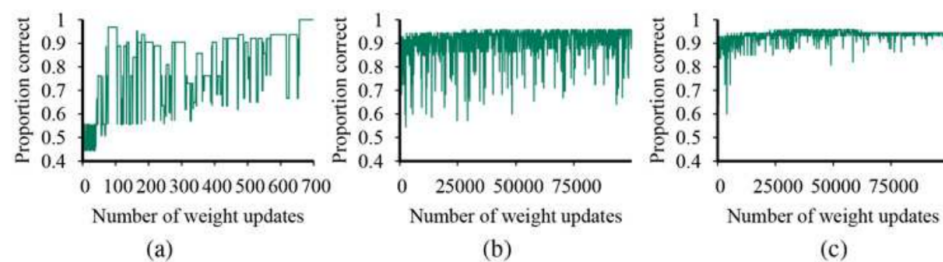
here,

- $\mathbf{h}_{\mathbf{W}}(\mathbf{x})$ represents the hypothesis for the classification.
- Z is the equation of decision boundary.
- $\mathbf{W}$ is the weight vector.
- $\mathbf{X}$ is the value of the input features, $X_1, X_2 \dots X_m$.
- W_0 is the bias term.

6.1.1 Perceptron Learning

Since, the result of $\mathbf{h}_{\mathbf{W}}(\mathbf{x})$ is either 0 or 1, we cannot choose the $\mathbf{W}$ minimizing loss. The gradient in the weight space is 0 almost everywhere except the point where $W \cdot X = 0$. So

Figure 19.16



(a) Plot of total training-set accuracy vs. number of iterations through the training set for the perceptron learning rule, given the earthquake/explosion data in Figure 19.15(a). (b) The same plot for the noisy, nonseparable data in Figure 19.15(b); note the change in scale of the x-axis. (c) The same plot as in (b), with a learning rate schedule $\alpha(t) = 1000/(1000 + t)$.

gradient descent will not work in this case. To solve this problem we introduce **perceptron learning**. In this process the weights are only adjusted when the prediction is incorrect.

$$W_0 = W_0 + \alpha \cdot (y - \mathbf{h}_W(\mathbf{X})). \quad (6.3)$$

$$W_j = W_j + \alpha \cdot (y - \mathbf{h}_W(\mathbf{X}))X_j. \quad (6.4)$$

Typically, perceptron rule uses a randomly chosen example from the data to compute the updated weight.

A **training curve** plots how well the classifier is doing on the same training data as it keeps learning, one update at a time.

When Perceptron Learning Fails

Perceptron rule does not work well with linearly non-separable noisy data.

When the data are not linearly separable, there is no perfect line that can divide the two classes without mistakes. The perceptron algorithm is designed to adjust its weights whenever it makes a mistake. Since mistakes can never be fully avoided in this case, the perceptron keeps finding errors and keeps adjusting the weights forever, even if it often comes close to the best possible solution. This stops the perceptron rule from converging.

Plot **a** of figure ?? shows how the perceptron learning rule improves over time when it is trained on linearly separable data.

In this case, the curve shows that the perceptron eventually finds a perfect (zero-error) straight-line separator between the two classes.

However, the learning process is not smooth—it jumps around before finally getting it right.

The number of updates can vary between different training runs. Plot **b** and **c** shows that

the perceptron rule fails to converge even after 10,000 updates when the dealing with noisy data.

Learning Parameter for Perceptron Rule

A decaying learning rate helps because, in the beginning, the algorithm can take big steps to explore quickly, and later, it takes smaller, finer steps to carefully settle near the best solution without overshooting. This gradual slowdown allows the perceptron to stabilize instead of endlessly bouncing around.

Plot c of ?? shows the training process when using a decaying learning rate. It still doesn't reach perfect convergence even after 100,000 updates, but it does much better than when using a fixed learning rate.

Normally, the perceptron rule may not converge to a stable solution if the learning rate stays fixed.

However, if we make the learning rate smaller over time — for example, using $\alpha = \frac{1000}{1000+t}$ (where t is the number of updates) — then the perceptron can be shown to converge to a minimum-error solution, as long as the training examples are presented in a random order.

Still, finding the true minimum-error solution is a very hard problem (NP-hard), so we expect that it will take many passes over the data before convergence happens.

6.1.2 Linear Classifier with Logistic Regression

In linear classification with logistic regression, instead of using hard thresholds for classification, we use a sigmoid function as the classifier.

$$\mathbf{h}_{\mathbf{w}}(\mathbf{Z}) = \frac{1}{1 + e^{-\mathbf{Z}}}, \quad (6.5)$$

where,

$$Z = \mathbf{W} \cdot \mathbf{X} + W_0. \quad (6.6)$$

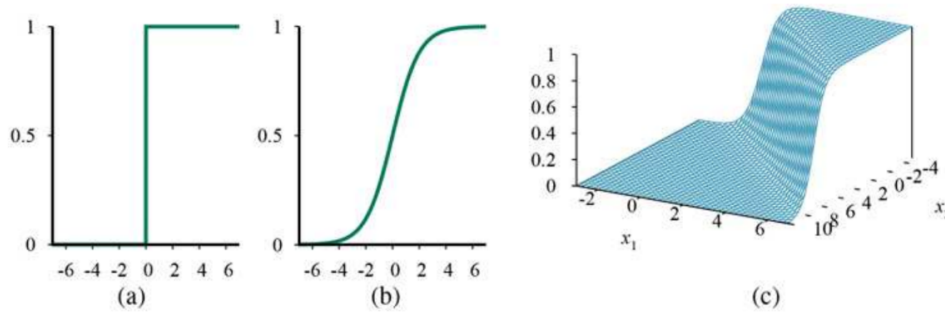
Here,

- W_0 is the bias term.
- X is the input vector with inputs $X_1, X_2 \dots X_m$.
- W is the weight vector with $W_1, W_2 \dots W_m$

The sigmoid function outputs the probability that an example belongs to Class 1.

The probability of belonging to Class 0 is simply the complement of this probability.

Unlike the hard threshold in 6.2, logistic regression uses a continuous function for its model, $\mathbf{h}_{\mathbf{w}}(\mathbf{X})$.



(a) The hard threshold function $Threshold(z)$ with 0/1 output. Note that the function is nondifferentiable at $z = 0$. (b) The logistic function, $Logistic(z) = \frac{1}{1+e^{-z}}$, also known as the sigmoid function. (c) Plot of a logistic regression hypothesis $h_w(\mathbf{x}) = Logistic(\mathbf{w} \cdot \mathbf{x})$ for the data shown in [Figure 19.15\(b\)](#).

Figure 6.1

Therefore, logistic regression minimizes a continuous loss function (MSE or binary cross-entropy), which is smooth and differentiable.

This allows the use of gradient descent, leading to more stable and gradual updates.

In contrast to perceptron learning, Logistic regression is able to find best-fit boundary even with noisy linearly non-separable data.

6.1.3 Gradient Descent for Logistic Regression

Since, we now use a continuous function for our hypothesis, we can implement the gradient descent algorithm to optimize the weights. The update rules will be as follows:

$$w_0^{(t+1)} = w_0^{(t)} - \alpha \frac{\partial \mathcal{L}}{\partial w_0} \quad (6.7)$$

$$w_1^{(t+1)} = w_1^{(t)} - \alpha \frac{\partial \mathcal{L}}{\partial w_1} \quad (6.8)$$

6.1.4 Computing Gradient

We will show the computation of gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$ for two types of Loss function.

- Mean squared Error, $L_2 = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$.
- Binary Cross-Entropy, $L = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \ln \hat{y}^{(i)} + (1 - y^{(i)}) \ln (1 - \hat{y}^{(i)})]$.

Here, $\hat{y} = \mathbf{h}_w(\mathbf{Z}) = \frac{1}{1+e^{-\mathbf{Z}}}$

6.1.5 Logistic regression: Computing Gradient for Mean Squared Error

Remember,

$$Z = W_0 + W_1X_1 + W_2X_2 \dots W_mX_m$$

So,

$$\frac{\partial Z}{\partial W_0} = 1 \quad (6.9)$$

and

$$\frac{\partial Z}{\partial W_j} = X_j \quad (6.10)$$

Where, X_j is the j 'th input and W_j is the associated weight.

Computing $\frac{\partial \hat{y}}{\partial W_0}$:

$$\begin{aligned} \frac{\partial \hat{y}}{\partial W_0} &= \frac{\partial}{\partial W_0} \left(\frac{1}{1 + e^{-z}} \right) \\ &= \frac{-1}{(1 + e^{-z})^2} \frac{\partial}{\partial W_0} (1 + e^{-z}) \\ &= \frac{-1}{(1 + e^{-z})^2} \frac{\partial(e^{-z})}{\partial W_0} \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \frac{\partial z}{\partial W_0} \\ &= \frac{1}{(1 + e^{-z})} \frac{e^{-z}}{(1 + e^{-z})} \frac{\partial z}{\partial W_0} \\ &= \frac{1}{(1 + e^{-z})} \frac{e^{-z} + 1 - 1}{(1 + e^{-z})^2} \\ &= \frac{1}{(1 + e^{-z})} \left(1 - \frac{1}{1 + e^{-z}} \right) \frac{\partial z}{\partial W_0} \\ &= \frac{1}{(1 + e^{-z})} \left(1 - \frac{1}{1 + e^{-z}} \right) \quad [\text{using 6.9}] \end{aligned}$$

$$\frac{\partial \hat{y}}{\partial W_0} = \hat{y}(1 - \hat{y}) \quad (6.11)$$

By similar calculation and using 6.10:

$$\frac{\partial \hat{y}}{\partial W_j} = \hat{y}(1 - \hat{y})X_j \quad (6.12)$$

Computing the gradient of Loss, Mean Squared Error

$$\frac{\partial \mathcal{L}}{\partial W_0} = \frac{\partial}{\partial W_0} \sum_{i=1}^m \frac{1}{m} (y^{(i)} - \hat{y}^{(i)})^2$$

using chain rule of differentiation,

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_0} &= -\frac{2}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) \frac{\partial}{\partial W_0} (y^{(i)} - \hat{y}^{(i)}) \\ &= -\frac{2}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) \frac{\partial \hat{y}}{\partial W_0} \end{aligned}$$

Similarly,

$$\frac{\partial \mathcal{L}}{\partial W_j} = -\frac{2}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) \frac{\partial \hat{y}}{\partial W_j} \quad (6.13)$$

using 6.11 and 6.12, we get,

$$\frac{\partial \mathcal{L}}{\partial W_0} = -\frac{2}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) \hat{y}^{(i)} (1 - \hat{y}^{(i)}) \quad (6.14)$$

and

$$\frac{\partial \mathcal{L}}{\partial W_j} = -\frac{2}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) \hat{y}^{(i)} (1 - \hat{y}^{(i)}) X_j \quad (6.15)$$

Finally, we can write the update rule in the gradient descent algorithm for logistic regression with mean squared error loss.

$$W_0^{(t+1)} = W_0^{(t)} + \alpha \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) \hat{y}^{(i)} (1 - \hat{y}^{(i)}) \quad (6.16)$$

and

$$W_j^{(t+1)} = W_j^{(t)} + \alpha \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) \hat{y}^{(i)} (1 - \hat{y}^{(i)}) X_j \quad (6.17)$$

6.1.6 Logistic Regression: Computing gradient of Binary Cross-Entropy

The Loss function with Binary Cross-Entropy is given by

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \ln \hat{y}^{(i)} + (1 - y^{(i)}) \ln (1 - \hat{y}^{(i)})] \quad (6.18)$$

Now,

$$\frac{\partial \mathcal{L}}{\partial W_0} = -\frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial W_0} [y^{(i)} \ln \hat{y}^{(i)} + (1 - y^{(i)}) \ln (1 - \hat{y}^{(i)})] \quad (6.19)$$

Computing $\frac{\partial}{\partial W_0} (y \ln \hat{y})$

$$\begin{aligned} \frac{\partial}{\partial W_0} (y \ln \hat{y}) &= \frac{y}{\hat{y}} \frac{\partial \hat{y}}{\partial W_0} \\ &= \frac{y}{\hat{y}} \cdot \hat{y}(1 - \hat{y}) \quad [\text{Using 6.11}] \end{aligned}$$

$$\frac{\partial}{\partial W_0} (y \ln \hat{y}) = y - y\hat{y} \quad (6.20)$$

Computing $\frac{\partial}{\partial W_0} ((1 - y) \ln (1 - \hat{y}))$

$$\begin{aligned} \frac{\partial}{\partial W_0} ((1 - y) \ln (1 - \hat{y})) &= \frac{1 - y}{1 - \hat{y}} \frac{\partial (1 - \hat{y})}{\partial W_0} \\ &= -\frac{1 - y}{1 - \hat{y}} \frac{\partial \hat{y}}{\partial W_0} \\ &= -\frac{1 - y}{1 - \hat{y}} \cdot \hat{y}(1 - \hat{y}) \quad [\text{Using 6.11}] \end{aligned}$$

$$\frac{\partial}{\partial W_0} ((1 - y) \ln (1 - \hat{y})) = y\hat{y} - \hat{y} \quad (6.21)$$

Using 6.20 and 6.21 in 6.19 we get

$$\frac{\partial \mathcal{L}}{\partial W_0} = -\frac{1}{m} \sum_{i=1}^m y^{(i)} - \hat{y}^{(i)} \quad (6.22)$$

A similar calculation using 6.12 we get,

$$\frac{\partial \mathcal{L}}{\partial W_j} = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) X_j \quad (6.23)$$

Finally, using 6.22 and 6.23 we get the update rule for gradient descent algorithm in logistic regression with binary cross-entropy.

$$W_0^{(new)} = W_0^{(old)} + \alpha \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) \quad (6.24)$$

$$W_j^{(new)} = W_j^{(old)} + \alpha \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)}) X_j \quad (6.25)$$

6.2 Example: Logistic Regression with Binary Cross-Entropy

We consider the dataset:

x_1	x_2	y
0.5	1.2	0
1.0	0.8	0
1.5	1.3	1
2.0	1.7	1

The logistic regression model is:

$$z = w_0 + w_1 x_1 + w_2 x_2, \quad \hat{y}_i = \frac{1}{1 + e^{-z}}$$

Initial weights: $w_0 = 0$, $w_1 = 0$, $w_2 = 0$. from the table:

Table 6.1: Gradient Computation at Initial Weights

x_1	x_2	y	z	$\hat{y}$	$\hat{y} - y$	$(\hat{y} - y)x_1$	$(\hat{y} - y)x_2$
0.5	1.2	0	0	0.5	0.5	0.25	0.60
1.0	0.8	0	0	0.5	0.5	0.50	0.40
1.5	1.3	1	0	0.5	-0.5	-0.75	-0.65
2.0	1.7	1	0	0.5	-0.5	-1.00	-0.85
Total:					0.0	-1.00	-0.50

$$\sum_{i=1}^6 (y - \hat{y}) = 0;$$

$$\sum_{i=1}^6 (y - \hat{y})x_1 = -1;$$

$$\sum_{i=1}^6 (y - \hat{y})x_2 = -0.50.$$

The gradients are:

$$\begin{aligned}W_0^{new} &= W_0^{old} + \alpha \frac{\partial \mathcal{L}}{\partial w_0} \\&= 0 + \alpha \cdot \frac{1}{4} \cdot 0 = \boxed{0} \\W_1^{new} &= W_1^{old} + \alpha \frac{\partial \mathcal{L}}{\partial w_1} \\&= 0 + \alpha \cdot \frac{1}{4} \cdot (-1.00) = \boxed{-.25} \\W_2^{new} &= W_2^{old} + \alpha \frac{\partial \mathcal{L}}{\partial w_2} \\&= 0 + \alpha \cdot \frac{1}{4} \cdot (-0.50) = \boxed{-.125}\end{aligned}$$

where α is the learning rate.